REAL-TIME OBJECT DETECTION AND COUNTING USING YOLO

Complete Viva Presentation Script, Project Explanation
and Cross-Question Preparation

Prepared from the project source code and bundled model checkpoints.

Includes: a ready-to-speak presentation, feature walkthrough, system architecture, YOLO explanation, model and dataset facts, technology stack, demo guide, limitations, alternatives, and likely viva questions.

Accuracy note: This project uses pretrained models for inference. It does not include a custom training dataset or a training run. The video count is a sum of detections across frames, not a count of unique tracked objects.

Use the quoted passages as your speaking script. Presenter notes are reminders and do not need to be read aloud.

1. Opening and project summary (about 1 minute)

“Good morning, ma'am. My project is titled Real-Time Object Detection and Counting Using YOLO. It is a web application that applies pretrained YOLO models to a webcam stream, uploaded images, local video files, and public YouTube video URLs. For object detection, it identifies object classes, draws bounding boxes, displays confidence scores, and calculates class-wise detection counts. In webcam mode, a separate pose model estimates human body keypoints, and simple geometric rules label a few poses such as hands raised, arms spread, and crouching. The backend is built with Python and Flask, OpenCV handles image and video processing, and the browser interface uses HTML, CSS, and JavaScript. We use pretrained weights; this project does not train a model from scratch. The main contribution is integrating detection, media processing, statistics, and a usable web interface.”

Transition

“I will first explain the problem and features, then the system architecture and YOLO pipeline, and finally the model, dataset, limitations, and possible improvements.”

2. Problem statement, objective, and scope

“The problem addressed is how to make object detection easy to demonstrate and use without requiring a user to run separate command-line scripts for each image or video. The objective is to accept common media inputs, run a pretrained detector, and present understandable visual results and basic counts in a browser.”

Key project objectives

- Detect common objects and estimate their locations in an image or video frame.
- Support webcam, still image, local video, and YouTube URL workflows.
- Display annotated output, class labels, confidence values, count summaries, and a live FPS estimate.
- Demonstrate frontend-to-backend integration with an inference pipeline.

“This is an educational prototype and integration project. It is not a custom model-training study or a production deployment.”

3. Features and user flow

Webcam detection

“When I start the camera, the browser requests a stream from the Flask server. The server opens camera device zero through OpenCV, reads frames, runs object detection and pose estimation, annotates each frame, and streams it back as a sequence of JPEG images. The browser separately polls the statistics endpoint to update object counts, FPS, and pose labels. The stop button turns off the stream and resets the displayed statistics.”

Image upload

“For an image, I select a PNG, JPG, JPEG, or WEBP file and submit it. The backend checks the extension, reads the image with OpenCV, runs object detection once, draws the detections, saves the annotated image, and returns its URL and statistics as JSON.”

Local video upload

“For a local video, the backend opens the file with OpenCV, reads it frame by frame, runs the detector on each frame, writes the annotated frames into an output MP4, and aggregates the frame-level detections. Supported extensions include MP4, AVI, MOV, MKV, and WEBM.”

YouTube URL processing

> “The application also accepts a public YouTube URL. It validates the URL, downloads a compatible stream using yt-dlp, and processes up to the first twenty seconds. This feature depends on network access and on a compatible video being available.”

Pose estimates

> “In webcam mode only, the pose checkpoint estimates up to seventeen body keypoints for detected people. Our Python code draws visible skeleton connections and applies geometric rules to label three target poses. This is not a trained activity-recognition classifier, and the pose labels are approximate.”

Statistics and UI

> “The interface shows per-class object counts, total detections, and FPS for the live feed. It displays the processed image or video, and gives status or error messages for workflows.”

4. System architecture and data flow

The application follows a simple browser-client and Flask-server architecture. Processing and model inference happen on the machine running the Flask application.

Architecture layers

- **Presentation layer:** HTML defines the dashboard, CSS styles it, and JavaScript handles buttons, file forms, API calls, status text, and result display.
- **Application layer:** Flask serves the page, accepts uploads, validates requests, exposes statistics, and returns output URLs or a live stream.
- **Computer-vision layer:** OpenCV captures or decodes media, passes frames to the models, draws annotations, encodes JPEG frames, and writes processed video.
- **Inference layer:** Ultralytics loads the YOLO detection and pose checkpoints and runs predictions.

Flow diagram in words

> “The flow is: browser input, then a Flask route, then OpenCV media decoding, then YOLO inference, then filtering and annotation, then statistics or an output media file, and finally the result returns to the browser.”

Important routes

- **/** serves the dashboard.
- **/api/start_camera** and **/api/stop_camera** control the webcam stream state.
- **/video_feed** streams multipart JPEG frames.
- **/api/stats** returns the latest statistics as JSON.
- **/upload/image**, **/upload/video**, and **/process/youtube** process the corresponding media.
- **/outputs/<filename>** serves generated files.

Presenter note: The webcam is opened by OpenCV on the Flask host. In this project, that means the camera attached to the computer running the server, not an arbitrary remote visitor's browser camera.

5. YOLO explained in detail

What YOLO means

"YOLO stands for You Only Look Once. It is a family of object-detection models designed to predict object locations and classes efficiently. Instead of using a separate repeated region-proposal and classification pipeline for every possible crop, a YOLO model processes the image through its neural network and produces candidate detections in one inference pass."

Detection pipeline

- **1. Input and preprocessing:** The image is read as pixel data and resized for inference. This application configures the object detector with an image size of 640.
- **2. Feature extraction:** Learned neural-network layers transform pixels into visual features, from local edges and textures to higher-level patterns useful for recognizing objects.
- **3. Prediction:** The detection model produces candidate boxes, class scores, and confidence values for objects in the image.
- **4. Confidence filtering:** Candidates below the configured object confidence threshold of 0.25 are filtered out.
- **5. Non-maximum suppression:** Overlapping duplicate candidates are reduced using an IoU threshold configured as 0.45.
- **6. Annotation and summary:** The remaining detections are drawn as rectangles and labels. The backend groups detections by class and totals them.

Terms to explain clearly

- **Bounding box:** A rectangle around an object, represented in this code using corner coordinates x1, y1, x2, y2.
- **Class:** The category predicted for an object, such as person, car, or bottle.
- **Confidence:** A model score for a detection. It is not a guarantee of correctness or a calibrated probability in every context.
- **IoU:** Intersection over Union, the intersection area of two boxes divided by their union area. It measures overlap.
- **NMS:** Non-maximum suppression keeps stronger predictions and removes highly overlapping duplicates according to an IoU rule.
- **Inference:** Using trained model weights to make predictions. This is what our application performs; training is a different process.

Why YOLO fits this demo

"YOLO is a reasonable choice because the project needs an understandable and practical real-time detection pipeline. Ultralytics also provides a convenient Python API for loading weights and running predictions. The nano model is relatively lightweight for a demo. However, the best detector depends on the target hardware, required accuracy, object sizes, and latency. We have not benchmarked all alternatives in this project, so we do not claim YOLO is universally the best."

6. Models and dataset facts

Object detection checkpoint

"The object detector checkpoint is yolov8n.pt. The n indicates the nano-size variant. I verified the bundled checkpoint reports the detection task and 80 class names. The code loads it through Ultralytics and configures confidence 0.25, IoU 0.45, and image size 640."

Pose checkpoint

> “The webcam pose model is yolov8n-pose.pt. I verified the checkpoint reports the pose task, a person class, and a keypoint shape of seventeen points. The application uses confidence 0.35, image size 320, and a maximum of three pose detections per frame.”

Dataset and training statement

> “We did not collect or label a project-specific dataset, and there is no training script or training experiment in this repository. We use the supplied pretrained checkpoints for inference. The detector has 80 class labels, and the pose model estimates seventeen keypoints. We should not claim that we trained the model or report project-specific accuracy or mAP without an actual evaluation.”

Presenter note: If asked which exact dataset or split trained the checkpoint, do not guess from the app code. State that the repository does not document the exact checkpoint training provenance. Say it is a pretrained general-purpose checkpoint; verify the checkpoint documentation before naming a precise dataset version or split.

How custom training could be added

> “For a domain-specific extension, I would collect representative images, label object boxes or keypoints, split data into training, validation, and held-out test sets, fine-tune a suitable checkpoint, and compare it with the pretrained baseline using precision, recall, and mAP. I would also test it on realistic lighting, viewpoints, and occlusion.”

7. Counting and FPS: precise interpretation

Image counts

> “For one image, the count is the number of retained detection boxes per class in that image. If the image contains two detected people and one detected car, the summary reports those per-image detections.”

Video counts - an important limitation

> “For video, the current code adds detections from each processed frame. It does not associate detections between frames and does not track unique object identities. The same car appearing in many frames contributes repeatedly. Therefore, this is a frame-detection total, not a unique-car count.”

FPS

> “For the live stream, FPS is calculated approximately as one divided by the time spent processing a frame, including both object detection and pose inference. It is a processing estimate, not a full browser-to-browser latency measurement. Actual speed depends on the CPU or GPU, input resolution, scene, and workload.”

8. Technology stack and project files

- **Python:** Backend application and inference orchestration.
- **Flask:** HTTP routes, page rendering, upload handling, JSON responses, and streaming.
- **Ultralytics YOLO:** Pretrained object detection and pose models.
- **OpenCV:** Camera capture, image/video decoding, drawing, JPEG encoding, and video output.
- **NumPy:** Keypoint array operations and pose geometry calculations.
- **yt-dlp:** Downloading compatible YouTube media for processing.
- **HTML, CSS, JavaScript:** Frontend structure, appearance, events, fetch calls, polling, and results.
- **Pillow:** A declared imaging dependency used in the Python environment/ecosystem; it is not a primary processing step in the main app flow.

Repository structure

- **app.py:** Flask app, model configuration/loading, detection, pose rules, media processing, and routes.
- **templates/index.html:** Dashboard elements, camera controls, statistics, and upload forms.
- **static/js/script.js:** Browser interactions, API calls, polling, and displaying the outputs.
- **static/css/style.css:** Responsive page styling.
- **yolov8n.pt and yolov8n-pose.pt:** Bundled pretrained model weights.
- **uploads/ and outputs/:** Input media and processed result storage.
- **requirements.txt:** Python package dependency declarations.

9. Alternatives and comparison

> “YOLO is not the only available detector. Faster R-CNN is a two-stage approach that can be considered when accuracy is prioritized, though it may have higher latency. SSD and MobileNet-SSD are lightweight alternatives for constrained hardware. RetinaNet is another one-stage detector. DETR-style models use transformer-based designs and have different compute and latency trade-offs. Classical OpenCV methods can be effective for narrowly defined visual tasks, but they are not a direct substitute for general pretrained object recognition.”
>
> “The right model depends on the application and must be measured. For this project, YOLO is a good fit because it offers practical inference speed, common pretrained models, and straightforward Python integration. We have not run a controlled benchmark against the alternatives, so I would not claim it outperforms every other model.”

Fair comparison criteria

- Measure precision, recall, and mAP on the same held-out data.
- Measure FPS or latency on the same computer, resolution, and software setup.
- Compare memory use, deployment constraints, object-size performance, and ease of maintenance.
- Use the target application requirements to decide whether speed, accuracy, size, or robustness matters most.

10. Limitations and future improvements

Current limitations

- Pretrained generic classes may not cover specialized objects or a specific local environment.
- No custom training or project-specific accuracy evaluation is included.
- Video detections are aggregated per frame; there is no tracking or unique-object count.
- Pose estimates use simple geometric thresholds and can be sensitive to occlusion, camera angle, and keypoint quality.
- Running two models in webcam mode can reduce FPS, especially on CPU-only hardware.
- The YouTube feature depends on connectivity and external video availability.
- The current Flask development entry point uses debug mode and binds to all interfaces. It is a demo configuration, not a production deployment configuration.

Possible next steps

- Add object tracking and unique counts using a tracker, such as ByteTrack or BoT-SORT.

- Allow model selection and fine-tune a model for a defined use case.
- Evaluate precision, recall, mAP, and measured latency on a documented test set.
- Improve pose rules or train/evaluate an activity classifier if activity recognition is a goal.
- Add cleanup and retention policies for uploaded and generated files.
- For deployment, use a production WSGI server, disable debug mode, configure trusted network access, and add appropriate upload and resource controls.

11. Live demonstration script (about 2 minutes)

Before the viva

- Run the application with python app.py and open http://127.0.0.1:5000/ on the Flask host.
- Check webcam permissions and lighting. Keep a few recognizable objects in view.
- Have a small test image and a short local video ready as backup. Keep network-dependent YouTube processing optional.

Suggested spoken demo

"I will demonstrate the webcam first. I click Start Camera. The server reads frames using OpenCV, runs the two pretrained models, and streams the annotated frames. We can see the object label and confidence above each bounding box. The statistics panel shows detections by class and the approximate processing FPS. If a person is visible, the pose panel displays the rule-based pose estimate."

"Next I will upload an image. The server processes this image once, saves an annotated result, and returns the detections and counts. For a video, the server processes frames one by one and creates an annotated output video. The video total represents detections across frames, not unique tracked objects."

If the webcam fails

"The webcam may be unavailable because of permissions, another application using it, or the camera being attached to a different machine. I can still demonstrate the image workflow, which exercises the same detector and annotation code on a static input."

12. Likely viva cross-questions and answers

Q1. What is the difference between classification and object detection?

"Classification assigns one or more labels to an image. Object detection predicts labels and locations, usually as bounding boxes, for each detected object."

Q2. Did you train the YOLO model?

"No. We use pretrained YOLOv8 checkpoints for inference. The repository contains no custom training dataset or training run. The project work is application integration and media processing."

Q3. What dataset did you use?

"We did not prepare a project-specific dataset. We use general pretrained checkpoints with 80 detector classes and a pose model with seventeen keypoints. The exact training provenance is not established by our application files, so I would verify checkpoint documentation before naming an exact dataset version or split."

Q4. What does confidence 0.25 mean?

“It is the detection threshold configured in the application. Predictions below it are filtered out. A lower threshold can retain more candidates but may add false positives; a higher threshold can reduce false positives but may miss valid objects.”

Q5. What is IoU and why use NMS?

“IoU measures the overlap between two boxes. Non-maximum suppression uses overlap to remove redundant candidate boxes for the same object, keeping stronger detections according to the configured threshold.”

Q6. Is the webcam opened in the browser?

“In this implementation, OpenCV opens camera device zero on the computer running Flask. The server then streams processed frames to the browser.”

Q7. Are pose labels learned by the pose model?

“The model predicts body keypoints. The labels hands raised, arms spread, and crouching are assigned by our own geometric rules in Python. We did not train a separate activity classifier.”

Q8. Does the video count represent unique objects?

“No. It sums detections frame by frame. A persistent object may be counted in many frames. Unique-object counting requires tracking IDs across frames, which is not implemented.”

Q9. Why could the FPS be low?

“The webcam pipeline runs both object and pose inference. Resolution, model size, CPU or GPU availability, and the number of objects affect processing time. Pose inference can reduce the rate further.”

Q10. Why choose YOLO instead of Faster R-CNN or SSD?

“YOLO is a practical fit for this real-time demo and is straightforward to integrate. Other models can be preferable under different accuracy, speed, and hardware requirements. We have not performed a controlled benchmark, so our choice is based on suitability rather than a universal claim of superiority.”

Q11. What metrics would you use to evaluate it?

“I would use precision, recall, and mean Average Precision on labeled held-out data for detection quality, and measure latency or FPS on a specified machine and resolution. I would report the test setup along with the numbers.”

Q12. Is the system production-ready?

“It is a prototype for demonstration. The development entry point has debug mode enabled and binds to all interfaces. A production version would require a production server, debug disabled, deployment controls, and additional validation and resource limits.”

Q13. What is the main contribution of your project?

“The contribution is integrating pretrained object and pose inference with live and uploaded media workflows, annotation, basic statistics, and a browser interface in one demonstrable application.”

Q14. What happens if the model file is missing?

“The application attempts to load the model through Ultralytics and raises a clear runtime error if loading fails. The model weights therefore need to be available locally or obtainable in the configured environment.”

Q15. What would you improve first?

“I would add tracking for unique counts, define a target use case and collect labeled data if general classes are insufficient, then evaluate accuracy and speed systematically on held-out examples.”

13. Closing statement

> “To conclude, this project demonstrates an end-to-end computer-vision application: media input, pretrained YOLO inference, frame annotation, counting, and browser presentation. Its strengths are the multiple input modes and clear integration. Its main limitations are the lack of custom training, lack of object tracking for unique video counts, and rule-based pose classification. Thank you. I am happy to answer questions.”

14. Last-minute memory sheet

- **Object model:** yolov8n.pt; detection task; 80 class names; confidence 0.25; IoU 0.45; image size 640.
- **Pose model:** yolov8n-pose.pt; pose task; 17 keypoints; confidence 0.35; image size 320; maximum 3 pose detections per frame.
- **Training:** none in this project; pretrained weights are used for inference.
- **Pose activities:** hands raised, arms spread, crouching; assigned by hand-written geometric rules from keypoints.
- **Video count:** frame-level detection sum, not unique objects; no tracker implemented.
- **Stack:** Python, Flask, Ultralytics, OpenCV, NumPy, yt-dlp, HTML, CSS, JavaScript.
- **Best positioning:** YOLO is a suitable real-time trade-off for this prototype, not universally the best model.
- **Do not invent:** custom training, exact dataset split, benchmark results, accuracy, mAP, or FPS claims that were not measured.